

Variadic lock_guard (Rev. 5)

Changes from P0156R0

In response to concerns about ABI breakage expressed in national body comments GB 61 and FI 8, the variadic lock guard construct proposed in this paper is given the new name `scoped_lock`. The wording below has been updated to reflect this.

Introduction

The basic idea of this proposal is that `std::lock_guard` would benefit from being variadic to support multiple locks analogously to how `std::lock` does.

`lock_guard` is a very useful and widely used way to manage the lifetimes of lock ownership.

```
std::mutex mtx;
void f(){
    std::lock_guard<mutex> lck(mtx); // Mutex will be unlocked however scope is exited
    /* ... */
}
```

Unfortunately, if more than one lock needs to be required, life gets unnecessarily complicated

```
void swap(MyType const &l, MyType const &r)
{
    std::lock(l.mtx, r.mtx);
    std::lock_guard<std::mutex> llck(l.mtx, std::adopt_lock);
    std::lock_guard<std::mutex> rlck(r.mtx, std::adopt_lock);
    /* ... */
}
```

This is a lot more advanced and error-prone than the single lock case. For example, you often see the deadlock-inviting

```
void swap(MyType const &l, MyType const &r)
{
    std::lock_guard<std::mutex> llck(l.mtx);
    std::lock_guard<std::mutex> rlck(r.mtx);
    /* ... */
}
```

or the exception-unsafe

```
void swap(MyType const &l, MyType const &r)
{
    std::lock(l.mtx, r.mtx);
    /* ... */
    l.mtx.unlock();
    r.mtx.unlock();
}
```

These are exactly the kinds of complexities that are easily avoided by `std::lock_guard` in the single-lock case. Wouldn't it be great if `std::lock_guard` was variadic so it also worked with multiple locks?

This paper proposes a class `scoped_lock` that does exactly that.

```
void swap(MyType const &l, MyType const &r)
{
    std::scoped_lock lck(l.mtx, r.mtx); // Leverages P0091R3, Template argument deduction for constructors
    /* ... */
}
```

This can work in the presence of shared locking as well (just like `std::lock` does) as shown in the following example shared by Howard Hinnant

```

#include <mutex>
#include <shared_mutex>

class X
{
    using Mutex = std::shared_timed_mutex;
    using ReadLock = std::shared_lock<Mutex>;

    mutable Mutex mut_;
    // more data
public:
    // ...
    X& operator=(const X& x)
    {
        if (this !=&x)
            ReadLock rl(x.mut_, std::defer_lock);
            std::scoped_lock lck(mut_, rl);
            // assign data ...
        }
        return *this;
    }
    // ...
};

```

Wording

Modify §30.4 [thread.mutex] as follows

```

template <class... MutexTypes> class lock_guard;
template <class... MutexTypes> class scoped_lock;

```

Revert §30.4.2.1 [thread.lock.guard] to match its form in the C++14 standard ISO/IEC 14882:2014.

Add a new section §30.4.2.x [thread.lock.scope] as follows

```

namespace std {
    template <class... MutexTypes>
    class scoped_lock {
    public:
        using mutex_type = Mutex; // If MutexTypes... consists of the single type Mutex

        explicit scoped_lock(MutexTypes&... m);
        explicit scoped_lock(MutexTypes&... m, adopt_lock_t);
        ~scoped_lock();

        scoped_lock(scoped_lock const&) = delete;
        scoped_lock& operator=(scoped_lock const&) = delete;
    private:
        tuple<MutexTypes&...> pm; // exposition only
    };
}

```

An object of type `scoped_lock` controls the ownership of lockable objects within a scope. A `scoped_lock` object maintains ownership of lockable objects throughout the `scoped_lock` object's lifetime (3.8). The behavior of a program is undefined if the lockable objects referenced by `pm` do not exist for the entire lifetime of the `scoped_lock` object. When `sizeof...(MutexTypes)` is 1, the supplied `Mutex` type shall meet the `BasicLockable` requirements. Otherwise, each of the mutex types shall meet the `Lockable` requirements. (30.2.5.2).

```
explicit scoped_lock(MutexTypes&... m);
```

Requires: If a `MutexTypes` type is not a recursive mutex, the calling thread does not own the corresponding mutex element of `m`.

Effects: Initializes `pm` with `tie(m...)`. Then if `sizeof...(MutexTypes)` is 0, no effects. Otherwise if `sizeof...(MutexTypes)` is 1, then `m.lock()`. Otherwise, then `lock(m...)`.

```
explicit scoped_lock(MutexTypes&... m, adopt_lock_t);
```

Requires: The calling thread owns all the mutexes in `m`.

Effects: Initializes `pm` with `tie(m...)`.

Throws: Nothing.

```
~scoped_lock();
```

Effects: For all `i` in `[0, sizeof...(MutexTypes))`, `get<i>(pm).unlock()`

Appendix: Example implementation

We implement using the `for_each_in_tuple` infrastructure [posted](http://stackoverflow.com/questions/16387354/template-tuple-calling-a-function-on-each-element) by Andy Prowl on Stack Overflow to lock and unlock all the mutexes in the tuple.

```
#include<mutex>
#include<tuple>
using std::tuple;
// Taken from http://stackoverflow.com/questions/16387354/template-tuple-calling-a-function-on-each-element
namespace detail
{
    template<int... Is>
    struct seq { };

    template<int N, int... Is>
    struct gen_seq : gen_seq<N - 1, N - 1, Is...> { };

    template<int... Is>
    struct gen_seq<0, Is...> : seq<Is...>{};

    template<typename T, typename F, int... Is>
    void for_each(T&& t, F f, seq<Is...>) {
        auto l = { (f(std::get<Is>(t)), 0)... };
    }
}

template<typename... Ts, typename F>
void for_each_in_tuple(std::tuple<Ts...> const& t, F f) {
    detail::for_each(t, f, detail::gen_seq<sizeof...(Ts)>());
}
// End of for_each_in_tuple implementation

template<typename ...Ts>
struct scoped_lock {
public:
    explicit scoped_lock(Ts&... ts) : mutexes(ts...) {
        lock(ts...);
    }

    scoped_lock(Ts&... ts, std::adopt_lock_t) : mutexes(ts...) {}

    ~scoped_lock() {
        for_each_in_tuple(mutexes, [](auto &m) { m.unlock(); } );
    }

    lock_guard(const lock_guard&) = delete;
    lock_guard& operator=(const lock_guard&) = delete;

private:
    tuple<Ts&...> mutexes;
};

template<typename T>
struct scoped_lock<T> {
public:
    typedef T mutex_type;

    explicit scoped_lock(T& _Mtx) : mtx(_Mtx) {
        mtx.lock();
    }

    scoped_lock(T& _Mtx, std::adopt_lock_t) : mtx(_Mtx) {}

    ~scoped_lock() {
        mtx.unlock();
    }

    scoped_lock(const scoped_lock&) = delete;
    scoped_lock& operator=(const scoped_lock&) = delete;

private:
    T& mtx;
};

template<>
struct scoped_lock<> {
    explicit scoped_lock() {}
    scoped_lock(std::adopt_lock_t) {}
};
```

```
~scoped_lock() {}  
scoped_lock(const scoped_lock&) = delete;  
scoped_lock& operator=(const scoped_lock&) = delete;  
};
```

Feature test macro

For the purpose of SG10, we recommend the feature test macro `__cpp_lib_scoped_lock`